

Serverless GPU — Setup Guide

How to turn on cheap, at-scale **original** product/category image generation using a serverless GPU on AWS. This is the setup companion to `IMAGE-COST-BUDGET.md` (cost) and the block in `backend/.env.example`. Not legal/financial advice; confirm current AWS prices before a large run.

Why serverless GPU

Paid image APIs (DALL·E, Midjourney) bill per image, which gets expensive across a large catalog. A serverless GPU on AWS runs an open/managed image model and bills either per-image (Bedrock) or per second of generation (SageMaker scale-to-zero) — no idle cost, no per-image list price. With the template-once design, an image is generated a single time and reused across all countries.

What it powers

- Product **template images** (one per product, reused per country) via `aiCatalogSeed`.
 - **Category tile images** via `aiCategoryImages`. Both go through one code path: `Core.GenerateImage` → `backend/sdk/integrations.ts`, wrapped by `backend/sdk/image-gen.ts` (which cost-guards, stores to S3 if configured, and falls back to text-only if unconfigured). AWS request signing is in `backend/sdk/aws/sigv4.ts`.
-

Option A — AWS Bedrock (recommended to start; true serverless, no infra)

You run nothing; AWS hosts the model and bills per image (fractions of a cent).

1. **Enable model access:** AWS Console → **Amazon Bedrock** → **Model access** → request/enable an image model in your region, e.g. **Amazon Titan Image Generator** (cheapest, ~\$0.01/img), **Amazon Nova Canvas** (~\$0.04/img), or **Stability SDXL**.
2. **Create IAM credentials** with permission `bedrock:InvokeModel` (an IAM user or role). Get the access key + secret.
3. **Set env variables** (Railway Variables or `backend/.env`):

```
IMAGE_PROVIDER=aws_bedrock
IMAGE_MODEL=amazon.titan-image-generator-v1      # or amazon.nova-canvas-v1:0 | stability.stable-diffusion-xl-v1
AWS_REGION=us-east-1                            # a region where
the model is enabled
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
```

4. Done. The catalog/category jobs generate images automatically, capped per run.

How the code calls it: a SigV4-signed POST to `bedrock-runtime.<region>.amazonaws.com/model/<modelId>/invoke`. Titan/Nova use `{taskType:"TEXT_IMAGE", textToImageParams:{text},`

imageGenerationConfig:{...}}; Stability uses {text_prompts:[...]}. The response's base64 image is returned as a data URL (then persisted to S3 if configured).

Option B — AWS SageMaker (your own model; scale-to-zero = cheapest at volume)

Best if you want full control (SDXL/FLUX) and the lowest per-image cost.

1. **Deploy a text-to-image model** to a SageMaker endpoint (e.g. a Hugging Face SDXL/FLUX container). Configure it as an **asynchronous / serverless-style endpoint that scales to zero** when idle, so you pay only while generating.

2. **IAM:** grant sagemaker:InvokeEndpoint.

3. **Set env variables:**

```
IMAGE_PROVIDER=aws_sagemaker
SAGEMAKER_IMAGE_ENDPOINT=your-endpoint-name
AWS_REGION=...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
```

4. **Container contract:** the code sends {"inputs": "<prompt>", "parameters": {}} and accepts either raw image bytes (image/*) or JSON with base64 (image / generated_image / artifacts[0].base64). If your container differs, adjust the aws_sagemaker branch in backend/sdk/integrations.ts.

Cost: an SDXL image on an L4/A10G-class GPU takes ~3-5 s; at ~\$1-1.50/GPU-hour that's roughly \$0.002/image, plus a little warm-up time — all images ≈ \$2-8.

Durable storage (optional but recommended)

If S3 is configured, generated images are uploaded and listings store a short URL instead of a big inline data URL:

```
S3_BUCKET=your-bucket
S3_PUBLIC_BASE=https://your-bucket.s3.us-east-1.amazonaws.com # or
a CDN URL
```

Without S3, images are embedded inline (still works, just larger rows).

Cost controls (already defaulted to the low-cost posture)

```
CATALOG_IMAGES_ENABLED=1          # 0 = launch text-only at $0
CATALOG_IMAGES_MAX_PER_RUN=100     # paces spend across scheduled
runs
CATALOG_SUBCATEGORY_IMAGES=0       # top-level tiles only (cheap)
CATALOG_IMAGE_SIZE=1024x1024
```

Plus AI_DAILY_SPEND_CAP_USD caps LLM text spend. All editable live in **Admin → Platform Settings**.

Verify it works

1. Set the env vars and restart the backend.
2. As admin, invoke `aiCategoryImages` once (or wait for the `daily-category-images` job).
3. Check that `CatalogCategory` rows get `image_url` values and category tiles show images.
4. Invoke `aiCatalogSeed`; confirm `MarketplaceListing` templates get `image_url`.

Troubleshooting

- **403 / SignatureDoesNotMatch (Bedrock):** the signer double-encodes non-S3 paths, which handles model ids containing : (e.g. `amazon.nova-canvas-v1:0`). If you still see it, verify `AWS_REGION` matches where the model is enabled and the IAM policy allows `bedrock:InvokeModel` on that model.
- **AccessDeniedException:** you didn't enable model access in Bedrock for that region, or the IAM policy is missing the action.
- **Empty images / text-only listings:** provider unconfigured or `CATALOG_IMAGES_ENABLED=0` — that's the safe fallback, not an error.
- **SageMaker returns unexpected shape:** adjust the response parsing in the `aws_sagemaker` branch of `integrations.ts` to match your container.

Where it lives in the code

- `backend/sdk/integrations.ts` — `GenerateImage` (`aws_bedrock` / `aws_sagemaker` / `openai` / `stability`).
- `backend/sdk/image-gen.ts` — cost guard, S3 persistence, text-only fallback.
- `backend/sdk/aws/sigv4.ts` — SigV4 signing (path double-encoding for non-S3).
- `backend/sdk/aws/s3.ts` — `uploadBytes` for storing generated images.
- Functions: `aiCatalogSeed`, `aiCategoryImages`. Settings: `IMAGE_*`, `CATALOG_IMAGE*` in `settings.ts`.